Transactions:

1. Transaction
2. ACID properties
3. State diagram of transaction
4. Shadow copy technique
5. Concurrent execution / serial execution
6. Serializability
   - Conflict
   - View
7. Schedules
8. Phantom problem
9. Cascading Rollbacks
10. Recoverable schedules

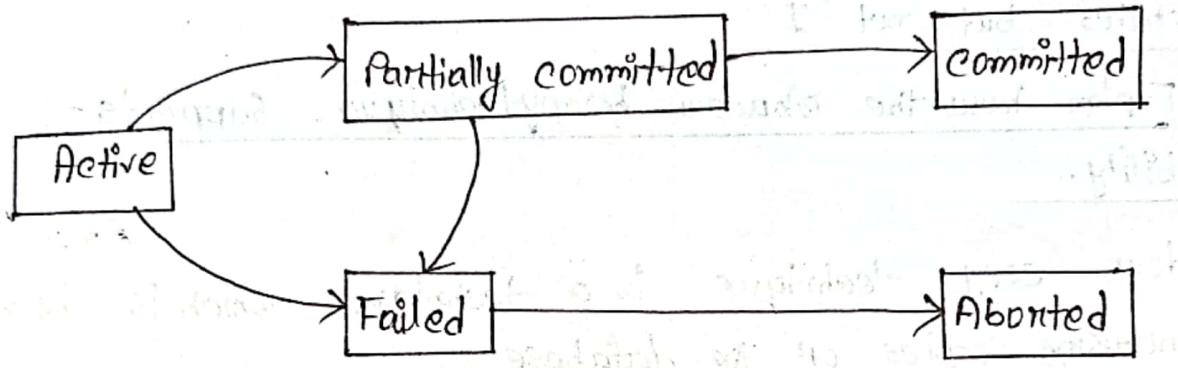
Transaction: A transaction can be defined as a group of tasks. A single task is the minimum processing unit which cannot be divided further. A transaction is a unit of program execution that accesses and possibly updates various data items. A transaction is initiated by a user program written in a high-level DML (SQL) or programming language (C++ or JAVA).

ACID Properties:

1. **Atomicity:** This property states that a transaction must be treated as an atomic unit that is either all of its operations are executed or none.
2. **Consistency:** The database must remain in consistent state after any transaction is executed.
   Execution of a transaction in isolation preserves the consistency of the database.
3. **Isolation:** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions.
   No transaction will affect the existence of any other transactions.
4. **Durability:** The database should be durable enough to hold all its updates even if the system fails or restarts.

After a transaction completes successfully the changes it has made to the database persist, even if there are system failures.

State diagram of transaction:



Active: The initial state, the transaction stays in this state while it is executing.

Partially committed: When a transaction executes its final operation, it is said to be in partially committed state.

Failed: A transaction is said to be failed if any of the checks made by the database recovery system fails. A failed transaction can no longer proceed further.

Aborted: After the transaction has been rolled back and the database restored to its prior to the start of the transaction.

Two operations after a transaction aborts:

1. Restart the transaction.
2. Kill the transaction.

Committed: If a transaction executes all its operation successfully, it is said to be committed.

How shadow copy technique helps ensuring 'ACD' Properties but not 'I'

OR, Explain how the shadow copytechnique supports durability.

shadow copy technique is a technique which is based on making copies of the database

⇒ Let us assume that only one transaction is active at a time.

⇒ A pointer db-pointer always points to the current consistent copy of the database.

⇒ All update are made on shadow copy of the database and db-pointer is made to point to the updated shadow copy only after the transaction reaches partial commit and all updated pages have been flushed to disk.

⇒ In case transaction fails, old consistent copy pointed to by db-pointer can be used and the shadow copy can be deleted.

Those procedures ensure Atomicity, Consistency, Durability but not isolation property.

Benefits.

Concurrent Execution over serial execution:

① Improved throughout and resource utilization:

A transaction consists of many steps. Some involve I/O activity, Others involve CPU activity. The CPU and the disks in a computer system can operate in parallel. I/O activity can be done in parallel with processor of the CPU.

② Reduced waiting time: There are mix transaction running on a system, some short and some long. If transactions run serially, a short transaction may have to wait for long preceding. It allows the short one to complete quickly.

In serial execution, a short transaction could get stuck behind a long one.

③ Increased scalability

④ Efficient resource utilization

⑤ Faster transaction processing

⑥ Enhanced system efficiency.

Example: Online shopping cart

Serial execution: Users add items to their shopping carts one by one.

Concurrent execution: Multiple users can simultaneously add items to their shopping carts.

Serializability: Serializability is a concept in database management systems that ensures the isolation and consistency of transactions. It is the classical concurrency scheme. It assumes that all access to the database are done using read and write operations.

Types: ① Conflict serializability
② view serializability

① Conflict Serializability: A schedule is said to be conflict serializable when the schedule is conflict equivalent to one or more serial schedules.
A schedule is conflict serializable if there exists an acyclic precedence graph for the schedule.

T_1	T_2
read (A)	
write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

② View serializability: It focuses on ensuring that the final state of the database. It is defined by a equivalence to a serial schedule with the same transaction such that respective transaction in the two schedules read and write the same data values.

Every conflict serializable schedule is also view serializable.

T_1	T_2
Read(Q)	
	write(Q)
write(Q)	

It is view serializability but not conflict serializability

Schedules: A schedule for a set of transactions must consist of all instructions of those transaction.

Schedule -1

Let T_1 transfer $50 from A to B, and T_2 transfer 10% of the balance from A to B.

(T_1 is followed by T_2)

T_1	T_2
read (A)	
A := A - 50	
write (A)	
read (B)	
B := B + 50	
write (B)	
commit	
	read (A)
	temp := A * 0.1
	A := A - temp
	write (A)
	read (B)
	B := B + temp
	write (B)
	commit

Schedule-2 (T_2 is followed by T_1)

T_1	T_2
	read (A)
	temp := A * 0.1
	A := A - temp
	write (A)
	read (B)
	B := B + temp
	write (B)
	commit
read (A)	
A := A - 50	
write (A)	
read (B)	
B := B + 50	
write (B)	
commit	

Schedule-3 Let T_1 and T_2 be the transactions defined previously. The following schedule is not a serial schedule but it is equivalent to schedule 1.

T_1	T_2
read (A)	
A := A - 50	
write (A)	
	read (A)
	temp := A * 0.1
	A := A - temp
	write (A)
read (B)	
B := B + 50	
write (B)	
commit	
	commit

read(B) B := B + temp write(B) commit

The sum 'A+B' is preserved.

Schedule-4 — The concurrent schedule does not preserve the sum of "A+B"

T_1	T_2
read(A)	
A := A-50	
	read(A)
	temp := A * 0.1
	A := A - temp
	write(A)
	read(B)
write(A)	
read(B)	
B := B+50	
write(B)	
commit	
	B := B+temp
	write(B)
	commit

Recoverable schedules:

T_1	T_2
read(A)	
write(A)	
	read(A)
	commit
read(B)	

Cascading Rollbacks: A single transaction failure leads to a series of transaction rollbacks.

T_1	T_2	T_3
read(A)		
read(B)		
write(A)		
	read(A)	
	write(A)	
		read(A)
Abort		

If T_1 fails, T_2 and T_3 must be rolled back.

Phantom problem: The term phantom problem refers to a phenomenon that can occur in the context of database transactions specially when dealing with concurrency control. Concurrency control ensures that multiple transactions. It specifically arises in situations where a transactions result set is affected by the insertion or deletion. by Concurrent transactions.